

Circle Creative Buro

ТЕСТОВОЕ ЗАДАНИЕ

Fullstack Developer — OpKit

React + NestJS + PostgreSQL + WebSocket

Мини-CRM с real-time обновлением задач

Контекст и цель

OrKit — продуктовая платформа для управления проектами и взаимодействия с клиентами. Нам нужен разработчик, который умеет строить продуктовые системы: правильная архитектура бэкенда, работа с real-time, чистый и читаемый код.

Тестовое задание имитирует реальные задачи, с которыми вы столкнётесь в первые недели работы.

Общие требования

Формат сдачи	GitHub репозиторий (публичный или доступ по ссылке)
Что должно быть в README	Инструкция по локальному запуску, описание структуры проекта, используемые технологии
Стек	React (TypeScript) + NestJS + PostgreSQL + Redis + WebSocket

Техническое задание

1. Аутентификация

Реализовать регистрацию и вход пользователя.

- POST /auth/register — email + пароль (bcrypt хеширование)
- POST /auth/login — возвращает JWT access token
- Защита всех остальных роутов через Guard
- Хранить данные пользователей в PostgreSQL

2. Управление задачами (CRUD)

Каждая задача принадлежит авторизованному пользователю.

- Поля задачи: id, title, description, status, created_at, updated_at
- Статусы: TODO → IN_PROGRESS → DONE
- GET /tasks — список всех задач пользователя
- POST /tasks — создать задачу
- PATCH /tasks/:id — обновить задачу или сменить статус
- DELETE /tasks/:id — удалить задачу
- Валидация через class-validator (пустой title — ошибка)

3. Real-time обновление через WebSocket

Это ключевой блок задания — именно здесь проверяется навык работы с real-time.

- Подключить WebSocket Gateway в NestJS (@WebSocketGateway)
- При изменении статуса задачи — отправить событие всем подключённым клиентам
- На фронте подписаться на события и обновлять список задач без перезагрузки страницы
- Событие должно содержать: id задачи, новый статус, timestamp

4. Фронтенд (React + TypeScript)

Интерфейс простой — главное чтобы он работал корректно.

- Страница регистрации и входа
- Страница со списком задач (канбан из 3 колонок или список с фильтром по статусу)
- Кнопки смены статуса у каждой задачи
- При получении WebSocket события — список обновляется автоматически
- Можно открыть два браузера, зайти с одного аккаунта — изменения видны в обоих

5. База данных

- PostgreSQL + Prisma ORM
- Схема в prisma/schema.prisma
- Миграции через prisma migrate
- Docker Compose для поднятия PostgreSQL локально — большой плюс

Критерии оценки

Критерий	Вес
Работает ли WebSocket (два браузера — оба обновляются)	Обязательно
JWT аутентификация реализована корректно	Обязательно
CRUD задач работает без ошибок	Обязательно
Структура проекта, читаемость кода	Высокий
TypeScript используется по-настоящему (нет any везде)	Высокий
Валидация входных данных	Средний
Docker Compose для БД	Плюс
Тесты (хотя бы unit на сервис)	Плюс

Что НЕ нужно делать

- Красивый UI — не цель, нам важна логика
- Деплой на сервер — только локальный запуск
- Сложные роли и права — один уровень доступа